StudyPilot

Stage 2 Implementation Report

EECS3311 | York University | Prepared November 3 2026

1 Project overview and implementation status

StudyPilot helps a student study imported course material. The application stores courses, source chunks, generated study content, quiz attempts, study plans and course-bound conversations in local JSON files. A Swing GUI and a command-line interface share AgentFacade and the same domain services.

All twelve Stage 1 features have implementation code and automated verification. The configured LLM is gpt-6.1-sol through an OpenAI-compatible Chat Completions service. The model generates summaries, concepts, questions, grades, explanations, study tasks and agent plans. Local Java code retrieves sources, validates structured output, executes tools and saves the actual results.

The current Java 17 clean build passed 307 JUnit 4 tests. The latest complete CLI workflow used thirteen actual model responses and separate JVM invocations to reopen saved work. The agent executed progress, weakTopics and quiz, resolved a topic from the preceding tool result and reused its saved conversation in a later invocation.

Delivery is still incomplete. Native GUI text entry was unreliable during the automated check, so a complete GUI walkthrough and native recording are not claimed. The CLI replays use captured application executions. Public video links and final student review remain pending. This report does not extend the confirmed Period 1 student review to later code.

Scope and limitations

The project uses plain Java 17, Swing, Maven Wrapper and JSON storage. PDFBox extracts PDF text; org.json maps saved objects; JUnit 4 supports tests. The five distinct patterns remain Facade, Strategy, Adapter, Factory Method and Observer. No feature or pattern has been silently replaced.

There is no OCR or embedding retrieval. Citation validation checks retrieved chunk IDs rather than proving semantic support for every sentence. A valid JSON grade can still contain an incorrect judgment. Study-plan context is limited to eight material chunks. JSON files use one writer per data directory and are not one database transaction. Recovery can restore supported counts and snapshots but cannot invent missing chronology. Full deterministic and KUMA evaluation remain Stage 3 work.

2 Feature to implementation mapping

Every entry below refers to the original feature ID in Stage1-Report.pdf. All listed Java files are under src/main/java/studypilot. The CLI replay references point to the local assets described in Section 7; uploaded timestamp links remain pending.

F01 Course/material import

Original F01 | UC01 / SD01 | Deterministic | Implemented

Files and roles: CourseService validates/extracts/chunks; CourseRepository saves source locations.

Key methods: createCourse, importMaterial, extractPdf. Patterns: Facade.

GUI or CLI input reaches AgentFacade and CourseService. The service validates the file, extracts text, retains original page or line locations, creates Chunk objects and saves the Course through CourseRepository. Invalid or duplicate imports add no material.

CLI replay A at 00:00.

F02 Summaries

Original F02 | UC02 / SD02 | AI | Implemented

Files and roles: SummaryService batches/merges/saves; OpenAIAdapter translates model calls.

Key methods: summarize, summaryPrompt, mergeSummaryPrompt, generate. Patterns: Facade, Adapter.

The facade selects the material and calls SummaryService. PromptTemplates builds context batches, LLMClient generates text, and the service checks all replies before replacing and saving the summary. Long materials require a final merge; a failed request preserves the earlier summary.

CLI replay A at 00:21.

F03 Concepts

Original F03 | UC03 / SD03 | Hybrid | Implemented

Files and roles: ConceptService merges and checks relationships; ResponseParser validates structured fields.

Key methods: extractConcepts, parseConcepts. Patterns: Facade, Adapter.

ConceptService receives the selected material, asks the model for structured concepts and uses ResponseParser to read them. It validates source IDs, merges duplicate names and removes invalid or cyclic parent links before saving the course.

CLI replay A at 00:28.

F04 Cited answers

Original F04 | UC04 / SD04 | Hybrid | Implemented

Files and roles: QuestionAnswerService retrieves; strategies score chunks; CitationValidator filters IDs.

Key methods: answer, retrieve, parseCitedAnswer, validate. Patterns: Facade, Strategy, Adapter.

QuestionAnswerService retrieves up to five chunks through the configured RetrievalStrategy. The model proposes an answer and source IDs. ResponseParser and CitationValidator retain only retrieved IDs and map them to file/page/line citations. Vague questions clarify; unmatched questions return without a model request.

CLI replay A at 00:38.

F05 Flashcards/review

Original F05 | UC05 and UC06 / SD03 and SD05 | Hybrid | Implemented

Files and roles: FlashcardService generates/reviews; Flashcard stores mastery; tracker notifies observers.

Key methods: generateCards, review, recordReview, getMastery. Patterns: Facade, Adapter, Observer.

FlashcardService validates generated fronts and backs, saves distinct cards and orders review by unreviewed state and mastery. A review saves the card counts and StudyEvent together in the course before updating ProgressTracker and notifying observers.

CLI replay A at 00:52.

F06 Quiz generation

Original F06 | UC07 / SD06 | Hybrid | Implemented

Files and roles: QuizService retrieves/generates; ResponseParser reads drafts; QuestionCreator and three subclasses validate typed products; QuizRepository saves immutable quizzes.

Key methods: createQuiz, quizPrompt, parseRawQuestions, buildQuestion, createQuestion, saveQuiz, loadQuiz. Patterns: Facade, Strategy, Adapter, Factory Method.

QuizService retrieves relevant material and requests drafts for the chosen types. Each matching QuestionCreator validates and builds a typed product. Bounded replacement keeps usable drafts. QuizRepository saves immutable questions and original IDs; the interfaces show requested and actual counts.

CLI replay B at 00:00.

F07 Adaptive difficulty

Original F07 | UC08 / SD06 | Deterministic | Implemented

Files and roles: QuizService chooses a level; ProgressTracker reads course/topic graded statistics; DifficultySuggestion holds the level, accuracy, count and reason; GUI/CLI accept an override.

Key methods: suggestDifficulty, chooseDifficulty, getAccuracy, getGradedCount. Patterns: Facade.

QuizService reads graded-only topic statistics through ProgressTracker. Fewer than five graded results use Medium. With enough results, accuracy below 50 percent chooses Easy, at least 80 percent chooses Hard, and the remaining range uses Medium. A valid explicit level overrides the suggestion.

CLI replay B at 06:20.

F08 Attempts/grading

Original F08 | UC09 and UC10 / SD07 and SD08 | Hybrid | Implemented

Files and roles: QuizAttempt preserves submitted answers; QuizGrader selects ExactMatchGrading or LLMRubricGrading; ResponseParser checks grades; QuizService resolves PENDING; QuizRepository restores attempts.

Key methods: startAttempt, submitAttempt, grade, parseGrading, regradePending, retryPendingGrading, getPendingQuestion, selfGrade, replacePendingResult. Patterns: Facade, Strategy, Adapter, Observer.

QuizService saves a fresh QuizAttempt, checks answer IDs and passes submission to QuizGrader. ExactMatchGrading handles objective answers; LLMRubricGrading handles nonempty short answers. Invalid model grades stay PENDING. Retry or explicit self-assessment resolves only pending short answers. Original answers and the first submission time remain fixed.

CLI replay B at 00:46.

F09 Weak topics

Original F09 | UC11 / SD09 | Hybrid | Implemented

Files and roles: ProgressTracker filters and sorts graded ConceptStats; WeakTopicAnalyzer requests a brief explanation; WeakTopicReport retains ranking and insufficient-data names on failure.

Key methods: getWeakTopics, analyze, getRankedStats, weakTopicPrompt. Patterns: Facade, Adapter.

ProgressTracker filters insufficient samples and sorts eligible ConceptStats by graded accuracy. WeakTopicAnalyzer requests an explanation of those actual counts. A missing key or failed explanation retains the deterministic ranking and states that generated advice is unavailable.

CLI replay B at 06:27.

F10 Study plans

Original F10 | UC12 and UC13 / SD10 and SD11 | Hybrid | Implemented

Files and roles: StudyPlanGenerator builds context and validates proposals; ResponseParser reads tasks; StudyPlan/StudyTask hold the dated schedule; StudyPlanRepository saves it; ProgressTracker records completion.

Key methods: createStudyPlan, generate, studyPlanPrompt, parseStudyTasks, validatePlan, getTasksForDay, getTotalMinutes, markTaskDone, recordTaskDone. Patterns: Facade, Adapter, Observer.

StudyPlanGenerator reads the course and graded-topic context, validates real dates and whole daily hours, and asks for a schedule. ResponseParser reads proposed tasks; local checks enforce dates, positive minutes, daily budgets and diagnostic-first work when evidence is insufficient. A valid plan is saved. Repeated completion of the same saved task counts once.

CLI replay C at 00:00.

F11 Progress/history

Original F11 | UC14 / SD12 | Deterministic | Implemented

Files and roles: ProgressTracker saves copied counts; Course retains review events; ProgressRestorer rebuilds selected-course counts/history; ProgressRepository backs up damaged files; HistoryRecorder/DashboardView observe updates.

Key methods: recordReview, recordQuizAttempt, recordTaskDone, restore, restoreSummary, recordEvent, getSummary, update, getHistory. Patterns: Facade, Observer.

AgentFacade restores the selected course before returning its progress. ProgressRestorer rebuilds supported review, submitted-attempt, graded-result and completed-task counts from saved sources. ProgressTracker and HistoryRecorder save copied values before exposing them. Damaged derived files are backed up; missing history is restored only as supported events or labelled snapshots.

CLI replay B at 06:34.

F12 Natural-language agent

Original F12 | UC15 / SD13 and SD14 | AI | Implemented

Files and roles: AgentPlanner proposes steps; AgentController validates and executes; ToolManager dispatches six service-backed tools; MemoryManager/MemoryRepository save messages; AgentResult holds actual results and trace.

Key methods: runAgent, handleRequest, createPlan, replan, executionPlanPrompt, parseExecutionPlan, validatePlan, resolveStep, executeWithRetries, execute, decide, loadSession, addMessage, toJson. Patterns: Facade, Adapter.

AgentController loads the course-bound session, asks AgentPlanner for structured steps, and validates and executes them through ToolManager. The six tools call the existing services. Whole-value references use actual successful result data. Each plan allows eight steps, a failed action at most two extra agent retries, and a request at most two replans. Memory stores the original request and actual response. Earlier successful outputs survive later failures.

CLI replay C at 00:48.

3 Implemented design patterns

The five distinct patterns match the Stage 1 choices. Grading uses Strategy in addition to retrieval; it is a second use of the same pattern and is not counted as a sixth pattern.

Facade

The GUI and CLI need one entry point without knowing every service. AgentFacade is the facade; SwingGUI and CLI are clients. Methods such as importMaterial, createQuiz, createStudyPlan and runAgent delegate to the domain services. DashboardView also reads through the facade. Course IDs keep every operation scoped to the selected course.

UML sources: docs/uml/class-view-1.uxf. Matching images are in docs/uml/images.

Strategy

QuestionAnswerService is the retrieval context and holds RetrievalStrategy. KeywordRetrieval and TfIdfRetrieval implement retrieve. AppConfig selects the algorithm. QuizService reuses that interface for source retrieval. QuizGrader is another context: ExactMatchGrading and LLMRubricGrading implement GradingStrategy.grade. Algorithm selection stays separate from the rest of each workflow.

UML sources: docs/uml/class-view-3.uxf; docs/uml/class-view-4.uxf. Matching images are in docs/uml/images.

Adapter

LLMClient is the application interface. OpenAIAdapter implements generate and converts requests and provider replies. OpenAIApiClient sends the HTTP request and records available usage. The implementation uses Chat Completions rather than the Stage 1 Responses API wording. Services still depend on LLMClient. MockLLMClient implements the same interface only in tests and is excluded from the application JAR.

UML sources: docs/uml/class-view-2.uxf. Matching images are in docs/uml/images.

Factory Method

QuestionCreator is the abstract creator. Its buildQuestion method performs shared checks and calls protected createQuestion. MCQQuestionCreator, TrueFalseQuestionCreator and ShortAnswerQuestionCreator override that factory method and build the corresponding Question products. QuizService selects a creator by QuestionType and does not construct those products directly. This follows the Stage 1 textbook structure.

UML sources: docs/uml/class-view-4.uxf. Matching images are in docs/uml/images.

Observer

Subject is the abstract subject; ProgressTracker extends it. Observer defines update, and HistoryRecorder and DashboardView implement it. Saved review, quiz and task events notify those observers. Stable event IDs and saved result keys prevent duplicate counting. Course-backed review events and recovery snapshots refine storage reliability without replacing the pattern.

UML sources: docs/uml/class-view-5.uxf. Matching images are in docs/uml/images.

Updated UML

The editable UMLet sources are in docs/uml. The eight class views cover the types in all 88 production source files, the private SwingGUI.StudyWorker and selected members. The test client is explicitly marked test only. The use-case view preserves UC01-UC15; SD01-SD14 reflect the current course scope, grading, recovery and agent behavior. Original Stage 1 files remain unchanged.

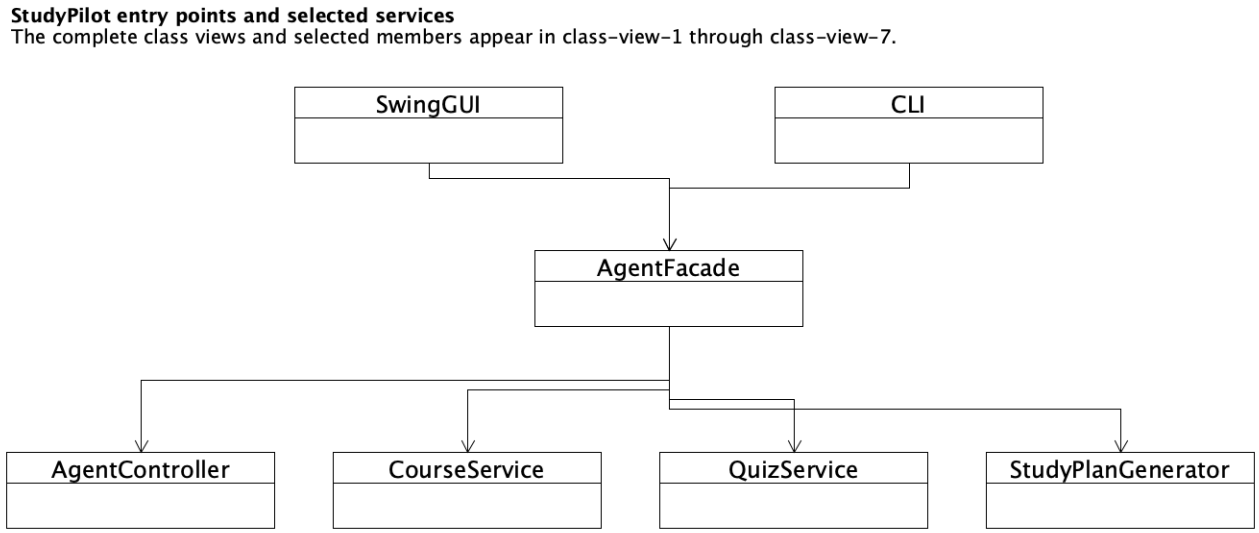


Figure 1 Application structure. The detailed UMLet views contain selected members, pattern roles and repository relationships.

4 Changes and rationale

The implementation retains all original features and all five original pattern choices. The consolidated changes below refine the original responsibilities and error paths. No unrelated product feature was added.

F01 source extraction

Implemented change: Per-page PDF extraction, Unicode-safe chunks and strict source identities.

Reason: Keep citations tied to original pages or lines and reject conflicting saved coordinates. Impact: UC01 and SD01; CourseService.extractPdf/splitIntoChunks and CourseRepository.checkMaterialIds/wholeNumber.

LLM connection

Implemented change: OpenAI-compatible Chat Completions behind LLMClient.

Reason: Use the selected compatible API while keeping service code independent of the wire format. Impact: F02-F04, F06, F08-F10 and F12; OpenAIAdapter/OpenAIApiClient; Adapter retained.

F02 prompt helpers

Implemented change: mergeSummaryPrompt for long materials.

Reason: Summarize batches and merge only after usable replies; preserve a saved summary on failure. Impact: UC02 and SD02; SummaryService and PromptTemplates.

F04 cited answer

Implemented change: CitedAnswer supports clarification; citation validity is derived from the citation list.

Reason: Represent ambiguity without unsupported citations and avoid a duplicate validity field. Impact: UC04 and SD04; QuestionAnswerService, ResponseParser and CitationValidator.

Configuration and storage

Implemented change: AppConfig, JsonFiles and explicit per-object JSON mappings.

Reason: Keep credentials local, validate settings and share temporary-file writes with atomic replacement where supported. Impact: Infrastructure and repository class view; no added design pattern.

F06 quiz sources and storage

Implemented change: Reuse RetrievalStrategy, balance selected types, bound replacement and preserve immutable saved quiz IDs.

Reason: Ground generation in imported material and reopen the same questions after restart. Impact: UC07 and SD06; QuizService, QuestionCreator subclasses and QuizRepository; Strategy and Factory Method retained.

F07 and F09 statistics

Implemented change: Course-scoped ProgressSummary owns topic counts and saved result keys.

Reason: Exclude PENDING data, require minimum samples and keep deterministic ranking available when explanation fails. Impact: UC08, UC11, SD06 and SD09; QuizService, ProgressTracker and WeakTopicAnalyzer.

F08 attempts and grading

Implemented change: Explicit course scope, displayed option numbers, stored attempts and pending-only resolution.

Reason: Avoid ambiguous objective answers, preserve the first submission, and count a resolved result once. Impact: UC09/UC10 and SD07/SD08; QuizAttempt, QuizGrader, ResponseParser.parseGrading and QuizRepository.

F10 plan validation

Implemented change: Real-date checks, 1-24 whole daily hours, three proposals and diagnostic-first schedules.

Reason: Reject unusable dates/budgets and avoid claiming weak topics without enough evidence. Impact: UC12/UC13 and SD10/SD11; StudyPlanGenerator.validatePlan/markTaskDone and StudyPlanRepository.

F05 and F11 history

Implemented change: Stable StudyEvent IDs, saved review events and copied derived writes.

Reason: Preserve actual events before later writes and retry observer persistence without duplicate counts. Impact: UC06/UC14 and SD05/SD12; FlashcardService, ProgressTracker, HistoryRecorder and ProgressRestorer; Observer retained.

F11 restart recovery

Implemented change: Rebuild supported counts and missing history from saved sources; retain unreadable files and explicit snapshot limits.

Reason: Recover derived state without inventing grades, transitions or timestamps. Impact: ProgressRestorer.restore/rebuildSummary/restoreHistory and ProgressRepository backups.

F12 planning and execution

Implemented change: courseId/stepId signatures, six real tools, result references, bounded retry/replan recovery and saved sessions.

Reason: Prevent foreign-course access, invalid dependencies and repeated completed actions while retaining earlier output. Impact: UC15 and SD13/SD14; AgentController, AgentPlanner, ToolManager, MemoryManager and MemoryRepository.

CLI agent input

Implemented change: Reject --json, --course or --session used instead of a request.

Reason: Stop an omitted request before planning or saving a failed turn. Ordinary text containing option names remains valid. Impact: CLI.agentCommand; C20 input and structured-result tests. Public service signatures and pattern choices stay the same.

C21 verification

Implemented change: Complete study and agent workflows across fresh facade instances.

Reason: Check source identity, grading, feedback, task completion, memory and derived-state recovery together. Impact: StudyWorkflowRestartTest uses the existing test-only LLM client with the real services and repositories. No new production dependency or API was required.

5 AI tools and collaboration log

Codex was used to generate and revise code, prepare tests and troubleshoot errors. The recorded development model is gpt-6.1-sol. The application API checks also report gpt-6.1-sol. The usage figures below distinguish recorded development-counter windows from measured application responses.

Student contribution and review

I chose the study-assistant scope, reduced the feature list to twelve, reviewed the design drafts and selected the compatible API/model connection. I clarified that weak-topic statistics use graded quiz answers while flashcard reviews are recorded separately. I read, reviewed, integrated and validated the Period 1 implementation.

My confirmed implementation review ends at C07, commit 3705839cf0db69bb21acb4da10070212d3aeba7c. Review of the Period 2 and Period 3 additions is pending. Codex executed the automated tests and CLI checks reported here.

Representative interactions

F01-F05 implementation

The implementation request used the Stage 1 responsibilities and required connected GUI and CLI workflows. Codex generated service/storage code, tests and troubleshooting changes. The confirmed student review checked responsibilities, integration and the Period 1 results. See docs/development-log.md.

Period 2 consistency review

The student requested a complete review of the seven Period 2 commits against Stage 1, Stage 2 and project wording. The review found stale documentation about courseId/stepId and the C14 commit link; those notes were corrected in C15. The review did not turn later unseen implementation into student-reviewed code.

C20 input defect

A new CLI test expected a usage error when an option occupied the request position. It initially received a planning failure instead. Codex added the small CLI.agentCommand guard and reran the clean suite and packaged CLI checks. The final C20 suite passed 304 tests.

C21 complete workflow

Codex added three restart workflow tests using the existing test-only LLM client and real services/repositories. An initial assertion used the wrong capitalization for a progress label; correcting that fixture assertion produced 307 passing tests. A separate production-JAR run used thirteen real API responses. A malformed authored answers file was rejected before submission and then corrected; this was not an application defect.

Recorded development usage

Recorded window	Input tokens	Output tokens	Total tokens
Initial implementation	18,889,846	129,769	19,019,615
Readability revision	6,908,744	67,171	6,975,915

The initial counter window ran October 24 from 21:21:40 to 22:09:35 Toronto time. The readability window ran October 24 from 23:18:49 to 23:43:13. Cached input and reasoning output are subsets, not additional tokens. Additional development usage after those windows is unavailable, so these figures are not a complete project-wide total. Development money cost is unavailable.

Measured application usage

Check window	Replies	Input	Output	Total
Recorded checks before C21	30	239,749	4,483	244,232
C21 production CLI checks	13	61,723	1,827	63,550
Known measured total	43	301,472	6,310	307,782

Counts come from completed service responses, including the earlier direct connection probe. The earlier timeout and two unsuccessful C14 summary attempts supplied no usage record and are excluded. Unknown usage is not treated as zero. Application money cost is unavailable because billing records were not returned. docs/usage-checkpoint.json retains the measured windows without credentials, prompts or raw conversations.

UMLet 15.1 exports the diagrams. python-docx and bundled LibreOffice format and render the report. HyperFrames renders the captured terminal replays. These presentation tools do not supply application responses or replace the required running GUI demonstration.

6 Development and verification summary

The following links identify meaningful implementation and test commits in the Toronto-local course periods. Each period requires at least five. The period summaries under docs retain the commit mappings. Actual Git dates were used.

October 20 to 26

C01 cb66fe2 feat: add application startup and safe local configuration

Committed Oct 25 00:37:22 EDT; push checked Oct 25 00:37:24 EDT. Clean test count 8.

C02 d9ebbe1 feat: import text and Markdown course materials

Committed Oct 25 02:02:33 EDT; push checked Oct 25 02:02:38 EDT. Clean test count 18.

C03 e476562 feat: import PDF materials with page citations

Committed Oct 25 03:47:04 EDT; push checked Oct 25 03:47:12 EDT. Clean test count 21.

C04 f2177e9 feat: connect LLM services and generate material summaries

Committed Oct 25 05:32:09 EDT; push checked Oct 25 05:32:16 EDT. Clean test count 31.

C05 c807572 feat: extract and validate course concepts

Committed Oct 25 07:16:57 EDT; push checked Oct 25 07:17:04 EDT. Clean test count 35.

C06 2722a4b feat: answer course questions with validated citations

Committed Oct 25 08:47:19 EDT; push checked Oct 25 08:47:28 EDT. Clean test count 42.

C07 3705839 feat: add flashcard review and study event observers

Committed Oct 25 10:03:27 EDT; push checked Oct 25 10:17:43 EDT. Clean test count 52.

October 27 to November 2

C08 f307408 feat: generate mixed quizzes with factory methods

Committed Oct 27 06:42:26 EDT; push checked Oct 27 06:42:28 EDT. Clean test count 73.

C09 22156e6 feat: create quiz attempts and grade objective answers

Committed Oct 27 08:09:25 EDT; push checked Oct 27 08:09:27 EDT. Clean test count 93.

C10 a6aeaa2 feat: grade short answers and resolve pending results

Committed Oct 28 14:24:58 EDT; push checked Oct 28 14:25:01 EDT. Clean test count 110.

C11 9658d77 feat: suggest quiz difficulty from graded performance

Committed Oct 28 15:44:52 EDT; push checked Oct 28 15:44:55 EDT. Clean test count 130.

C12 4848e82 feat: identify weak topics and explain study priorities

Committed Oct 28 17:15:42 EDT; push checked Oct 28 17:15:50 EDT. Clean test count 147.

C13 46da209 feat: generate and complete validated study plans

Committed Oct 28 19:06:40 EDT; push checked Oct 28 19:06:48 EDT. Clean test count 170.

C14 91097fa feat: execute study agent plans with tools and memory

Committed Oct 28 20:34:17 EDT; push checked Oct 28 20:34:24 EDT. Clean test count 202.

November 3 to 9

C15 9fe9487 feat: recover agent execution within retry and replan limits

Committed Nov 03 05:48:41 EST; push checked Nov 03 05:48:50 EST. Clean test count 224.

C16 4a6b0bc feat: restore progress history and prevent duplicate counting

Committed Nov 03 07:23:50 EST; push checked Nov 03 07:24:14 EST. Clean test count 245.

C17 c4226b1 test: cover material and quiz persistence workflows

Committed Nov 03 08:27:45 EST; push checked Nov 03 08:27:56 EST. Clean test count 262.

C18 3c346f4 test: cover grading difficulty and study plan integration

Committed Nov 03 10:01:50 EST; push checked Nov 03 10:02:02 EST. Clean test count 276.

C19 4dd147b test: cover agent recovery and conversation isolation

Committed Nov 03 11:31:03 EST; push checked Nov 03 11:31:13 EST. Clean test count 291.

C20 5789f86 test: verify CLI commands and structured agent results

Committed Nov 03 13:15:51 EST; push checked Nov 03 13:16:00 EST. Clean test count 304.

C21 adds complete restart workflows in StudyWorkflowRestartTest.java. The file history links to the commit containing those tests. Workflow test history

Current verification

The latest ./mvnw -q clean verify used Java 17 and passed 307 tests with zero failures, errors or skips. C21 added three complete workflows: ordinary restart, rebuilding derived progress/history after their removal in a test directory, and retaining earlier work after later generation failures. Scripted replies remain under src/test and do not enter the application JAR.

The production CLI checks ran summaries, concept extraction, cited answering, flashcard generation/review, mixed quizzes, pending grading and real retries, graded-only feedback, study plans, repeated task completion and a result-dependent agent plan. The final separate-process read showed one review, five submitted attempts, thirty graded answers, six correct answers and one completed task. Four empty submissions deliberately created verification history; those counts are not student study performance.

Important corrected defects include split Unicode characters, unverified citation IDs, coerced saved numbers or flags, conflicting source locations, duplicate derived counts after interrupted writes and missing CLI requests reaching the planner. Storage checks preserve damaged source bytes and retain backups. The original Stage 1 PDF remains unchanged.

Remaining checks and responsibilities

Complete the native GUI walkthrough and real GUI recording. Review the later implementation and demonstrations. Verify a fresh GitHub clone against the final published source and assets. Upload the videos so the instructor and teaching assistants can access them, then replace the pending video links with actual timestamp links and check them. The student submits the public repository URL to the course system. No course-system submission is performed by this work.

7 Demonstration index

The three CLI assets replay the captured inputs and output of actual packaged application executions. The replays retain real IDs and results, identify model involvement and shorten network waits. They are supporting CLI evidence. The required native GUI recording and uploaded accessible video URLs are still pending.

Feature	Segment	Local CLI asset	Accessible URL
F01 Course/material import	A 00:00	demo/recording-A-cli.mp4	Upload link pending
F02 Summaries	A 00:21	demo/recording-A-cli.mp4	Upload link pending
F03 Concepts	A 00:28	demo/recording-A-cli.mp4	Upload link pending
F04 Cited answers	A 00:38	demo/recording-A-cli.mp4	Upload link pending
F05 Flashcards/review	A 00:52	demo/recording-A-cli.mp4	Upload link pending
F06 Quiz generation	B 00:00	demo/recording-B-cli.mp4	Upload link pending
F07 Adaptive difficulty	B 06:20	demo/recording-B-cli.mp4	Upload link pending
F08 Attempts/grading	B 00:46	demo/recording-B-cli.mp4	Upload link pending
F09 Weak topics	B 06:27	demo/recording-B-cli.mp4	Upload link pending
F10 Study plans	C 00:00	demo/recording-C-cli.mp4	Upload link pending
F11 Progress/history	B 06:34	demo/recording-B-cli.mp4	Upload link pending
F12 Natural-language agent	C 00:48	demo/recording-C-cli.mp4	Upload link pending

How to review a segment

A covers imports, duplicate rejection, summaries, concepts, cited answers, unrelated questions, generated cards and remembered review. B covers invalid quiz counts, all three question types, malformed answer input, pending grades, real grading retry, repeated submissions, graded-only difficulty, weak topics and restored progress/history. C covers invalid dates, actual dated plans, repeated completion and the real agent trace, including a saved-context follow-up and missing-request validation.

Verification answers for the first attempt were taken from the saved model answers to exercise grading. Four later attempts intentionally submitted empty answer maps to create sufficient weak-topic data. The replay labels this setup. Those actions are demonstration inputs, not student assessment results.

Before submission, add the native GUI segment and the actual uploaded timestamp links to this index. Check every link without requesting access. Keep credentials and runtime JSON files local.